Estatly Guide: Dashboard

The Estatly dashboard explained file by file and line by line: how the data is fetched and shaped, how the page is composed, and how lists handle loading, empty and error states. With the decisions behind them, the changes made in review, interview questions and answers, and vocabulary.

Branch: refactor/dashboard. Updated November 6, 2026. Matches the code in your repo after this review.

Contents

- 1. The big picture
- 2. pages/Dashboard.tsx
- 3. useDashboard.ts
- 4. api/apiDashboard.ts
- 5. lib/counts.ts
- 6. PROPERTY_STATUSES in lib/constants.ts
- 7. DashboardLayout.tsx
- 8. Panel.tsx
- 9. StatStrip.tsx
- 10. StatusBars.tsx
- 11. DataRenderer.tsx
- 12. PendingList.tsx
- 13. ViewingList.tsx and ViewingGroup.tsx
- 14. DashboardSkeleton.tsx
- 15. Decisions and changes
- 16. Still open
- 17. Interview questions
- 18. Vocabulary
- 19. Self-test

1. The big picture

The dashboard is the first page every user sees after signing in. It answers one question: **what needs my attention right now?**

How the data flows

Layer	File	Job
Page	pages/Dashboard.tsx	Places the feature component
Component	DashboardLayout + small parts	Displays ready-made data
Hook	useDashboard	Gets the user, runs the cached query
API	getDashboardApi	Four parallel queries, counts and shapes the result
Database	Supabase + RLS	Returns only the rows each role may read

What each role sees

Role	Top of the page	Numbers
Admin, manager	Listings waiting for approval, next viewings	The whole agency
Agent	Their own viewings only (agents cannot approve)	Only their own listings, clients and viewings

The page in three layers of priority

- **1. Action first:** listings to approve and the next viewings.
- **2. Context numbers, kept small:** published, under offer, active clients, viewings in the next 7 days.
- **3. Breakdowns:** listings by status and clients by stage.

A design decision you can explain: the layout follows what the user must do, not what data exists.

2. pages/Dashboard.tsx

```
import DashboardLayout from "@/features/dashboard/DashboardLayout"

export default function Dashboard() {
  return <DashboardLayout />
}
```

- **Pages stay thin.** The route points to a page, and the page only places the feature component.
- All dashboard logic lives in features/dashboard, so everything about one feature is in one folder. That is a **feature-based folder structure**.
- Pages are what App.tsx lazy-loads, so keeping them thin keeps routing simple.

3. useDashboard.ts

```
export function useDashboard() {
  const { user } = useUser()
  const role = user?.user_profile?.role
  const userId = user?.id
```

- The dashboard needs to know who is asking: an agent sees only their own work.
- useUser reads the cached user, so this adds no extra request.

```
  const { data, isLoading, error, refetch } = useQuery({
    queryKey: ["dashboard", role, userId],
    queryFn: () => getDashboardApi(role!, userId!),
    enabled: Boolean(role && userId),
  })
```

- **The query key includes role and userId.** Each user gets their own cache entry, so an agent never sees the admin's cached numbers. Everything a query depends on belongs in its key.
- **enabled** makes this a **dependent query**: it waits until the user has loaded instead of running with undefined values.
- **role! and userId!** are non-null assertions, safe because enabled guarantees both exist when the query runs.
- queryFn is an arrow function, so the API is called with arguments only when TanStack Query decides to fetch.

```
  return { dashboard: data, isLoading, error, refetch, role }
}
```

- Renames data to dashboard, which says what it is.
- refetch is returned so the error message can offer Try again. role is returned because the layout changes for agents.

4. api/apiDashboard.ts

One function that runs four queries in parallel and returns everything the dashboard shows, already counted and shaped.

```
const today = startOfToday()
const inSevenDays = addDays(today, 7)
```

- Midnight today and midnight seven days later: the window for "viewings, next 7 days".
- date-fns replaced hand-written date math (setHours and 7 * DAY_MS). It was already in the bundle through react-day-picker.

```
let propertiesQuery = supabase
  .from("properties")
  .select(
    "id, title, price, listing_type, status, main_image, " +
    "agent:user_profiles!properties_agent_id_user_profiles_fkey" +
    "(full_name)"
  )
  .order("created_at", { ascending: true })
```

- **let, not const**, because the agent filter reassigns it below.
- **The query is not sent yet.** A Supabase query builder only runs when awaited, so you can keep adding conditions.
- select names only the columns needed. agent: is a **join** that fetches the agent's name through a foreign key in the same request. The !fkey part says which relationship to use, because two foreign keys link these tables.
- Oldest first: the listing that has waited longest for approval comes first. A product decision written in code.

```
let clientsQuery = supabase.from("clients").select("status")

let upcomingQuery = supabase
  .from("viewings")
  .select(
    "id, scheduled_at, client_id, " +
    "property:properties(id, title, main_image), " +
    "client:clients(full_name)"
  )
  .eq("status", "scheduled")
  .gte("scheduled_at", today.toISOString())
  .order("scheduled_at", { ascending: true })
  .limit(6)
```

- Clients: only the status column, because the dashboard only counts them.
- Upcoming viewings: scheduled, from today on, soonest first, at most 6. toISOString turns the Date into the format the database expects.

```
let nextWeekQuery = supabase
  .from("viewings")
  .select("id", { count: "exact", head: true })
  .eq("status", "scheduled")
  .gte("scheduled_at", today.toISOString())
  .lt("scheduled_at", inSevenDays.toISOString())
```

- **head: true returns no rows, only the count.** Downloading rows just to count them would be waste.
- gte and lt make a half-open range: from today, up to but not including day 8.

```
if (role === "agent") {
  propertiesQuery = propertiesQuery.eq("agent_id", userId)
  clientsQuery = clientsQuery.eq("assigned_agent_id", userId)
  upcomingQuery = upcomingQuery.eq("agent_id", userId)
  nextWeekQuery = nextWeekQuery.eq("agent_id", userId)
}
```

- **RLS is the security; these filters set the scope.** RLS lets agents read every property, so without this filter an agent's dashboard would count the whole agency.
- This is why the queries use let: each builder gets one more condition.

```
const [properties, clients, upcoming, nextWeek] = await Promise.all([
  propertiesQuery, clientsQuery, upcomingQuery, nextWeekQuery,
])

const failed =
  properties.error ?? clients.error ?? upcoming.error ?? nextWeek.error
if (failed) throw toRequestError(failed, "Dashboard")
```

- **Four requests in parallel**: the total time is about the slowest one, not the sum. The same pattern as the TinyMoviez dashboard.
- Supabase returns { data, error } instead of throwing, so Promise.all never rejects. That is why each error is checked.
- ?? chains pick the first error that exists. One failure fails the whole dashboard, because half the data would show wrong totals.

```
const propertyRows =
  (properties.data ?? []) as unknown as DashboardProperty[]
const upcomingRows =
  (upcoming.data ?? []) as unknown as DashboardViewing[]
```

- **as unknown as is a double cast.** The Supabase client is not typed with the database schema, so TypeScript does not know the shape of joined rows. The cast says "trust me" and switches off type checking (see Still open).

```
return {
  propertyCounts, clientCounts,
  pending: propertyRows.filter((p) => p.status === "pending-approval"),
  upcoming: upcomingRows,
  stats: {
    listings: propertyRows.length,
    published: countOf(propertyCounts, "published"),
    underOffer: countOf(propertyCounts, "under-offer"),
    activeClients: countOf(clientCounts, "active"),
    viewingsNextWeek: nextWeek.count ?? 0,
  },
}
```

- **The API shapes the data for the UI.** Components receive ready-made numbers and lists. Counting happens once here, not in five components.
- The DashboardData type describes this object, so every component knows exactly what it receives.

5. lib/counts.ts

```
export function countByStatus(
  rows: { status: string | null }[]
): StatusCount[] {
  const counts: Record<string, number> = {}
  for (const row of rows) {
    const status = row.status ?? "unknown"
    counts[status] = (counts[status] ?? 0) + 1
  }
  return Object.entries(counts).map(([status, count]) => ({
    status,
    count,
  }))
}
```

- The counting pattern from your rep sets: an object as a counter, (counts[x] ?? 0) + 1, like charCount.
- **Record<string, number>** is a TypeScript type for an object with string keys and number values.
- The parameter type only asks for a status field, so properties and clients both fit.
- Object.entries turns { published: 12 } into [["published", 12]], and map reshapes each pair into { status, count }.

```
export function countOf(rows: StatusCount[], status: string) {
  return rows.find((row) => row.status === status)?.count ?? 0
}
```

- Finds one status's count. ?.count ?? 0 returns 0 when that status has no rows.

Why these live in lib, not features: the API layer used to import them from features/dashboard. Dependencies should point down: features use api and lib, never the other way.

6. PROPERTY_STATUSES in lib/constants.ts

```
export const PROPERTY_STATUSES: {
  status: PropertyStatus
  label: string
  color: string
}[] = [
  { status: "pending-approval", label: "Pending approval",
    color: "var(--status-pending)" },
  { status: "published", label: "Published",
    color: "var(--status-published)" },
  ...
]
```

- The listing lifecycle in order, with a label and a color for each status. It sits next to its twin CLIENT_STAGES.
- Typed with PropertyStatus, so a typo like "publised" is a TypeScript error.
- Colors are CSS variables, so they follow the light and dark theme automatically.

7. DashboardLayout.tsx

```
const viewAllLink = (
  <Link to="/viewings" className="...">View all</Link>
)
```

- **A JSX element stored in a variable**, outside the component. JSX is just a value, so it can live in a constant. It is created once and used in two places.

```
const { dashboard, isLoading, error, refetch, role } = useDashboard()
const isAgent = role === "agent"
```

- One hook gives the component everything.
- isAgent is a **derived value**: computed during render, not stored in state. If you can compute it, do not store it.

```
{isLoading && <DashboardSkeleton isAgent={isAgent} />}
{error && <ErrorMessage error={error} onRetry={() => refetch()} />}
{dashboard && ( <> ... </> )}
```

- **The three states of any data screen:** loading, error, data. Each uses && conditional rendering.
- onRetry is an arrow function, so refetch runs on click, not during render.
- <> is a **fragment**: it groups elements without adding a div.
- No blank flash while the user loads: ProtectedRoute already waited for the user, so the query starts as soon as the page mounts.

```
{isAgent ? (
  <Panel title="Your viewings" action={viewAllLink}>
    <ViewingList viewings={dashboard.upcoming} />
  </Panel>
) : (
  <div className="grid grid-cols-1 gap-6
    lg:grid-cols-[minmax(0,2fr)_minmax(0,1fr)]">
    <Panel title="Waiting for approval"
      count={dashboard.pending.length}>
      <PendingList properties={dashboard.pending} />
    </Panel>
    <Panel title="Next viewings" action={viewAllLink}>
      <ViewingList viewings={dashboard.upcoming} />
    </Panel>
  </div>
)}
```

- **Role-based UI:** agents cannot approve listings, so they get no approval panel.
- A 2-to-1 column split on large screens. **minmax(0, ...)** is the Grid version of min-width: 0: grid columns will not shrink below their longest text by default, so a long title could push the page wider than the screen.

```
<StatStrip items={[
  isAgent
    ? { label: "Your listings", value: dashboard.stats.listings }
    : { label: "Published", value: dashboard.stats.published },
  { label: "Under offer", value: dashboard.stats.underOffer },
  ...
]} />
```

- **Data-driven UI:** pass an array of objects and the component decides how to show them. Only the first item changes by role.

```
<StatusBars config={PROPERTY_STATUSES}
  counts={dashboard.propertyCounts} empty={EMPTY_LISTINGS_BY_STATUS} />
<StatusBars config={CLIENT_STAGES}
  counts={dashboard.clientCounts} empty={EMPTY_CLIENTS_BY_STAGE} />
```

- **One component, two charts.** What changes (statuses, colors, counts) comes in as props. Reusability through configuration.

8. Panel.tsx

```
interface PanelProps {
  title: string
  count?: number
  action?: ReactNode
  className?: string
  children: ReactNode
}
```

- A props interface describes what the component accepts. ? makes a prop optional.
- **ReactNode** is anything React can render: text, an element, a list or null.
- **children** is whatever is placed between the tags: <Panel>this part</Panel>.
- **action** is a slot: the caller passes a whole element and Panel decides where it goes.

```
<section className={cn(
  "min-w-0 rounded-xl border bg-card p-4 sm:p-5",
  className
)}>
  <h2 className="min-w-0 truncate text-base font-semibold">
    {title}
    {count !== undefined && (
      <span className="ml-2 ... tabular-nums">{count}</span>
    )}
  </h2>
  {action}
  {children}
</section>
```

- **cn** merges classes. tailwind-merge inside it lets a caller's className override a default (p-8 replaces p-4).
- <section> with an <h2>: each panel is a labeled region for screen readers.
- **count !== undefined avoids a classic bug.** {count && ...} would print a literal 0 on the page, because 0 && x returns 0 and React renders numbers.
- **tabular-nums** gives every digit the same width, so numbers do not shift.

9. StatStrip.tsx

```
export default function StatStrip({ items }: { items: StatItem[] }) {
  return (
    <dl className="grid grid-cols-2 gap-px overflow-hidden rounded-xl
      border bg-border lg:grid-cols-4">
      {items.map((item) => (
        <div key={item.label} className="bg-card px-4 py-3">
          <dt>{item.label}</dt>
          <dd>{item.value}</dd>
        </div>
      ))}
    </dl>
  )
}
```

- An inline props type: fine for a single prop.
- **dl, dt, dd** is a description list, the semantic HTML for label and value pairs.
- **The gap-px trick:** the container's background is the border color, and the 1px gaps between cards show it as thin dividers. No double borders.
- 2 columns on phones, 4 on large screens, with one class.

10. StatusBars.tsx

```
const rows = config.map((item) => ({
  ...item,
  count: counts.find((c) => c.status === item.status)?.count ?? 0,
}))
```

- **It starts from config, not counts.** Every status appears in lifecycle order, even with 0. Mapping counts would drop empty statuses and lose the order.
- Spread plus one new field: the immutable pattern.

```
const max = Math.max(...rows.map((row) => row.count))
if (max === 0) return <EmptyMessage empty={empty} compact />
```

- Math.max takes separate arguments, so **spread** unpacks the array.
- An early return when everything is zero: there is nothing to compare, so it shows an empty state instead of empty bars.

```
<div aria-hidden="true" className="mt-1.5 h-2 rounded-full bg-muted">
  <div className="h-full rounded-full transition-[width] duration-300"
    style={{
      width: `${(row.count / max) * 100}%`,
      backgroundColor: row.color,
    }} />
</div>
```

- **Width relative to the largest, not the total.** The biggest status fills the bar, which makes differences easy to see.
- **Inline style for dynamic values:** Tailwind classes are fixed at build time, so a width or color from data must be an inline style.
- **aria-hidden**: the bar is decoration, because the number is written next to it.
- transition-[width] animates only the width.

11. DataRenderer.tsx

One component for the four states of every list. Used in 7 places in the app.

```
interface DataRendererProps<T> {
  data: T[] | undefined
  isLoading?: boolean
  error?: unknown
  loading?: ReactNode
  empty: EmptyState
  compact?: boolean
  onRetry?: () => void
  render: (data: T[]) => ReactNode
}
```

- **<T> makes it a generic component.** T is the type of item you pass. Pass properties, and render receives typed properties.
- **render is a render prop:** a function that receives the data and returns UI. DataRenderer decides when to call it; the caller decides what it draws.

```
if (isLoading) return <>{loading}</>
if (error) {
  return (
    <ErrorMessage error={error} onRetry={onRetry} compact={compact} />
  )
}
if (!data || data.length === 0) {
  return <EmptyMessage empty={empty} compact={compact} />
}
return <>{render(data)}</>
```

- **Guard clauses in a fixed order:** loading, error, empty, data. Every list handles all four states the same way, and no screen can forget the empty state.
- On the dashboard, the lists only pass data, empty and render, because DashboardLayout already handles loading and errors for the whole page.

Empty and error text lives in lib/states.ts (EMPTY_PENDING, EMPTY_UPCOMING_VIEWINGS...), so the wording stays consistent across the app.

12. PendingList.tsx

```
const MAX_ROWS = 5
const hidden = properties.length - MAX_ROWS
```

- A named constant instead of a magic number. hidden is a derived value: how many did not fit.

```
<DataRenderer data={properties} empty={EMPTY_PENDING} compact
  render={(items) => (
    <ul className="-mx-2">
      {items.slice(0, MAX_ROWS).map((property) => (
        <li key={property.id}>
          <Link to={`/properties/${property.id}`}
            className="flex min-h-14 items-center gap-3 ...">
```

- compact with no value means compact={true}.
- slice(0, 5) shows at most 5 without mutating the array.
- **key={property.id}**: the database id is the best key, unique and stable. The array index is a classic mistake when lists can reorder.
- The whole row is a link, and min-h-14 (56 px) keeps it easy to tap on phones.

```
{property.agent?.full_name ?? "No agent"} ·{" "}
<span className="text-highlight tabular-nums">
  {formatPrice(property.price, property.listing_type)}
</span>
```

- The join can return null, so optional chaining plus a fallback.
- {" "} is an explicit space: JSX removes whitespace at line breaks.

```
{hidden > 0 && (
  <Link to="/properties">{hidden} more in Properties</Link>
)}
```

- hidden > 0 is a boolean, so this && is safe. {hidden && ...} would print 0, the bug Panel avoids.

13. ViewingList.tsx and ViewingGroup.tsx

ViewingList

```
render={(items) => {
  const today = items.filter((v) => isToday(new Date(v.scheduled_at)))
  const later = items.filter((v) => !isToday(new Date(v.scheduled_at)))
  return (
    <div className="space-y-4">
      {today.length > 0 && (
        <ViewingGroup label="Today" viewings={today} />
      )}
      {later.length > 0 && (
        <ViewingGroup label="Upcoming" viewings={later} showDate />
      )}
    </div>
  )
}}
```

- This render prop has a body (curly braces and return), because it builds two groups first.
- date-fns isToday replaced a hand-written toDateString comparison.
- "Today" does not need dates; "Upcoming" does, so it passes showDate.

ViewingGroup

```
export default function ViewingGroup({
  label, viewings, showDate = false,
}: {
  label: string
  viewings: DashboardViewing[]
  showDate?: boolean
}) {
```

- **Extracted into its own file** during this review, so each component has one job: ViewingList splits the viewings, ViewingGroup draws one group.
- showDate = false is a **default parameter**, so callers only pass it when they need it.

```
<li key={viewing.id} className="flex min-h-14 items-center gap-3 py-2">
  <p className="... tabular-nums">
    {formatViewingTime(viewing.scheduled_at)}
  </p>
  {showDate && <p>{formatViewingDate(viewing.scheduled_at)}</p>}
  <p className="truncate">
    {viewing.property?.title ?? "Property removed"}
  </p>
  <p className="truncate">{getClientLabel(viewing)}</p>
</li>
```

- **"Property removed":** if the property was deleted, the join returns null. A real edge case, handled.
- **getClientLabel** shows "Client assigned to another agent" when RLS hides the client. The UI explains the security rule instead of showing "Unknown".

14. DashboardSkeleton.tsx

```
export default function DashboardSkeleton({
  isAgent,
}: {
  isAgent: boolean
}) {
  return (
    <div role="status" className="space-y-6">
      <span className="sr-only">Loading dashboard</span>
      {isAgent ? (
        <Skeleton className="h-72 rounded-xl" />
      ) : (
        <div className="grid grid-cols-1 gap-6
          lg:grid-cols-[minmax(0,2fr)_minmax(0,1fr)]">
          <Skeleton className="h-72 rounded-xl" />
          <Skeleton className="h-72 rounded-xl" />
        </div>
      )}
      ...
```

- A **skeleton** is a gray placeholder in the same shape as the real page, so nothing jumps when data arrives. That jump is called a **layout shift**.
- It now takes isAgent: agents get one top panel, everyone else two, matching the real layout. The role is known during loading because ProtectedRoute already waited for the user.
- role="status" with sr-only text: screen readers hear "Loading dashboard".

15. Decisions and changes

Use these as STAR stories: situation, task, action, result.

Fixing the dependency direction

My API file imported counting helpers from a feature folder, and the file was called constant even though it held functions. I moved the helpers to lib/counts.ts and the status list to lib/constants.ts next to its twin. Now dependencies only point down: features use api and lib, never the reverse.

One date library instead of hand-written math

The dashboard built date ranges with setHours and 24 * 60 * 60 * 1000, and checked today with toDateString. My date picker already ships date-fns, so I used startOfToday, addDays and isToday. The code reads like the rule it implements.

A skeleton that matched the wrong layout

The loading skeleton always showed two top panels, but agents see one, so the page jumped when data arrived. The role is already known while loading, so the skeleton takes isAgent and matches each role's real layout.

Splitting a component

ViewingList also contained ViewingGroup. I moved ViewingGroup into its own file, so ViewingList only splits viewings into today and upcoming, and ViewingGroup only draws a group.

Role-scoped data on top of RLS

RLS lets agents read all properties, so an agent's dashboard would have counted the whole agency. The API adds agent filters to every query: RLS is the security, the filters set the scope.

16. Still open

Item	Why it matters
as unknown as casts	The Supabase client is untyped. createClient<Database> with generated types would type every query and remove the casts
All property rows are fetched to count them	Fine for a demo agency. At scale, count in the database with a Postgres function or count queries
properties_by_status function is unused	getPropertiesByStatusApi is never called. Decide in the properties branch: use it or delete it
One error fails the whole dashboard	Reasonable for totals, but the viewings panel could still show if only the counts failed
Demo data	Viewings at 00:09 and 02:04 and "Jane Deo" look odd to recruiters

17. Interview questions

Q1. Walk me through how your dashboard loads its data.

The page renders DashboardLayout, which calls a useDashboard hook. The hook waits for the signed-in user, then runs one TanStack Query. Its function runs four Supabase queries in parallel with Promise.all, counts the results and returns one object with lists, counts and stats. The components only display it.

Q2. Why is the user's role in the query key?

Everything a query depends on belongs in its key. Agents and admins get different data, so each user needs their own cache entry. Otherwise an agent could briefly see the admin's cached numbers.

Q3. What is a dependent query?

A query that waits for other data before running. My dashboard query has enabled: Boolean(role && userId), so it only runs once the user has loaded.

Q4. Why Promise.all, and how do you handle errors with it?

The four queries are independent, so running them in parallel takes about as long as the slowest one. Supabase returns errors instead of throwing, so I check each result and throw the first error through toRequestError.

Q5. What does head: true do in Supabase?

It returns only the count, no rows. I use it for viewings in the next seven days, because downloading rows just to count them would be waste.

Q6. If RLS already protects the data, why filter by agent in the query?

RLS lets agents read every property, because the properties page shows them all. The dashboard should show an agent only their own work, so the filters set the scope. RLS is the security, the filters are the product logic.

Q7. What does the double cast as unknown as mean, and is it safe?

It tells TypeScript to trust me about a type it cannot check. My Supabase client is not typed with the database schema, so joined rows need it. It works, but it turns off checking. The proper fix is a client typed with generated database types.

Q8. Why does the API count and shape the data instead of the components?

So it happens once and components stay simple. They receive ready-made numbers and lists. It also keeps the logic in one testable place.

Q9. What is a render prop?

A prop that is a function returning UI. My DataRenderer handles loading, error and empty states, and calls render(data) only when there is data. The caller decides what to draw.

Q10. What is a generic component?

A component with a type parameter. DataRenderer<T> works with any item type, and TypeScript knows the exact type inside render.

Q11. Why check count !== undefined instead of count &&?

Because 0 && anything returns 0, and React renders numbers, so a count of zero would print a stray 0. Checking undefined shows a real 0 and hides only a missing count.

Q12. Why use the database id as a key instead of the index?

Keys tell React which item is which between renders. Indexes change when a list reorders or filters, which can mix up item state. Database ids are unique and stable.

Q13. Why does StatusBars start from the config instead of the counts?

So every status appears in lifecycle order, even with a count of zero. Mapping the counts would drop missing statuses and lose the order.

Q14. Why inline styles for the bar width when you use Tailwind?

Tailwind generates classes at build time, so it cannot create a width like 37.5% from data. Dynamic values go in style; fixed styling stays in classes.

Q15. What is a skeleton, and what is layout shift?

A skeleton is a placeholder in the shape of the real content. Layout shift is content jumping when data arrives. My skeleton matches each role's layout, so nothing moves.

Q16. What does minmax(0, 1fr) fix in CSS Grid?

Grid columns will not shrink below their longest content by default, so a long title can overflow the page. minmax(0, ...) lets the column shrink, and truncate adds the ellipsis.

Q17. Why dl, dt and dd for the stat strip?

It is the semantic HTML for label and value pairs, so screen readers read each label with its value.

Q18. How does your dashboard differ by role?

Admins and managers see listings waiting for approval and the whole agency's numbers. Agents cannot approve, so they see their own viewings and only their own numbers.

Q19. Why did you split ViewingGroup into its own file?

Single responsibility: ViewingList splits viewings into today and upcoming, ViewingGroup draws one group. Each file is small and has one reason to change.

Q20. What would you improve next on the dashboard?

Type the Supabase client with generated database types to remove the casts, and count statuses in the database instead of fetching every row, which matters as the agency grows.

18. Vocabulary

Term	Meaning
Feature-based structure	Folders by feature (dashboard, properties), not by file type
Dependent query	A query that waits for other data (enabled)
Query key	The cache identity of a query; includes everything it depends on
Join	Fetching related rows through a foreign key in one request
Derived value	Computed during render instead of stored in state
Conditional rendering	Showing UI only when a condition is true (&&, ternary)
Fragment	<></>: groups elements without an extra node
children	The content placed between a component's tags
Slot	A prop that takes a whole element, like action
Render prop	A function prop that returns UI
Generic component	A component with a type parameter: DataRenderer<T>
Description list	dl, dt, dd: label and value pairs
Skeleton	A placeholder in the shape of the real content
Layout shift	Content jumping when data arrives
Single responsibility	Each component or file has one job
Dependency direction	Features import from lib and api, never the reverse

19. Self-test

Say each answer out loud in under 45 seconds, without looking.

1. What does the dashboard show, and why in that order?
2. Walk me through how the dashboard data is loaded.
3. Why are role and userId in the query key?
4. What does enabled do in useDashboard?
5. Why are the queries built with let?
6. What does head: true do?
7. Why filter by agent when RLS exists?
8. What does as unknown as mean?
9. What does countByStatus do, step by step?
10. Why did the counting helpers move to lib?
11. What are the three states DashboardLayout handles?
12. What is a slot, and where does Panel use one?
13. Why count !== undefined?
14. What is a render prop?
15. Why start StatusBars from the config?
16. Why inline styles for the bar width?
17. What is a layout shift, and how did you prevent it?
18. Why did you extract ViewingGroup?
19. What would you improve next?